Selenium E2E testiranje

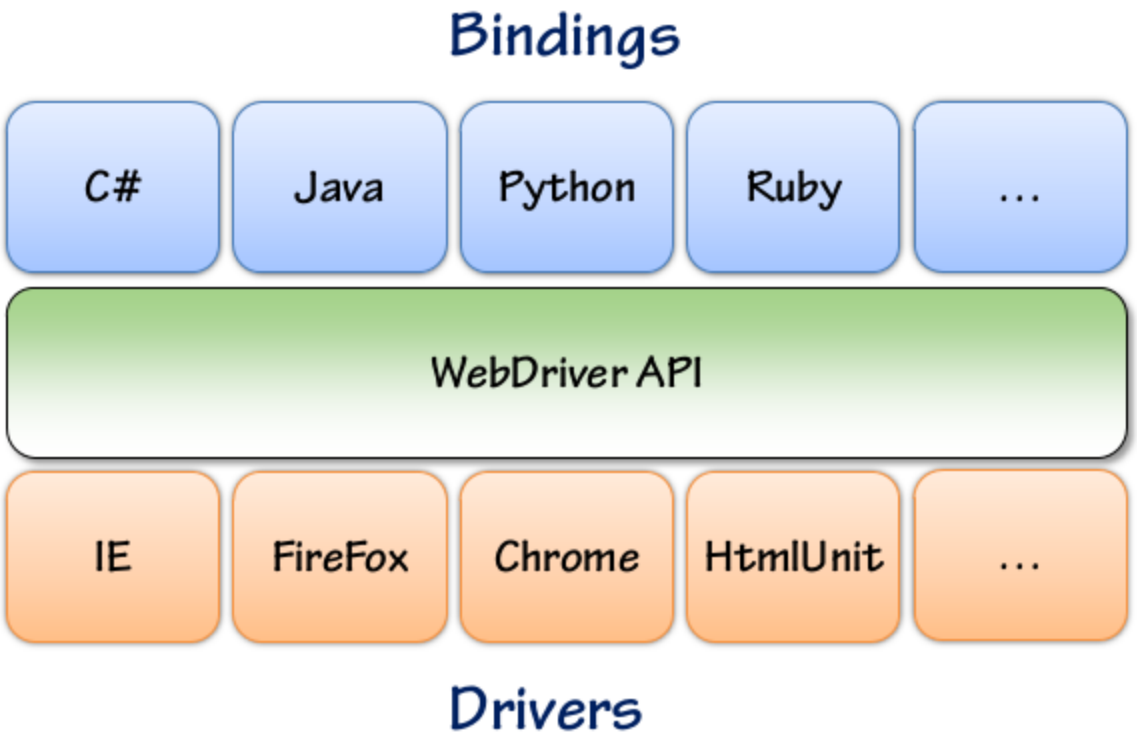
Autor: Goran Savić

Selenium

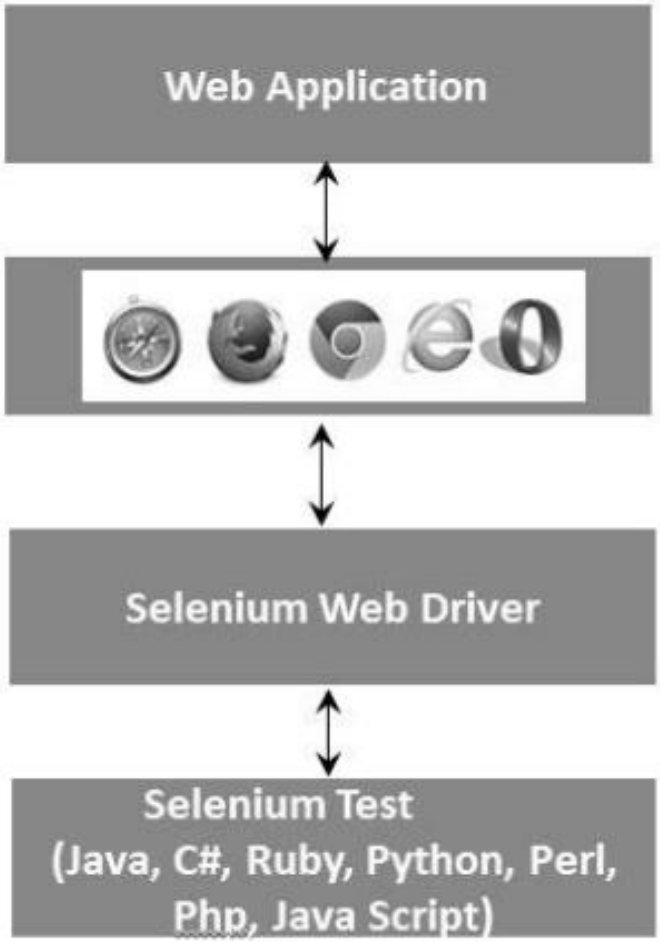


- Selenium je projekat koji sadrži različite biblioteke i alate za podršku automatizacije *end-to-end* testova koji se izvršavaju u *browseru*
- Selenium WebDriver
  - Automatizuje browser
  - Koristi se za automatizovane e2e testove
  - Omogućuje programsko izvršavanje akcija u *browseru*
  - Pruža API kroz koji mu se test kod može obratiti kako bi programski simulirao korisničke akcije u *browseru*
    - kod je moguće pisati u različitim programskim jezicima

Selenium WebDriver arhitektura



Mesto Selenium testova u arhitekturi veb aplikacije



Primer Selenium testa

- Programski se pristupa elementu HTML stranice
- Sprovodi se određena akcija nad elementom
  - klik na dugme
  - unos teksta
  - ...

```
WebElement tbName =
   browser.findElement(By.id("firstName"));
tbName.sendKeys("Dejan");
```

- Primer
  - `Primeri/selenium/StudentTest.java`

WebDriver komande

Komanda	Opis
driver.get("URL")	Navigacija na zadati URL
element.sendKeys("inputtext")	Unos teksta u input polje
element.clear()	Briše sadržaj input polja
select.deselectAll()	Deselektuje sve opcije na prvom SELECT elementu na stranici
select.selectByVisibleText("some text")	Selektuje zadatu opciju u SELECT elementu
driver.switchTo().window("windowName")	Stavlja fokus na drugi prozor u browseru
driver.switchTo().alert()	Pomaže u radu sa Alertom
driver.navigate().forward()	Za navigaciju napred
driver.navigate().back()	Za navigaciju nazad
driver.close()	Zatvara browser dodeljen driver-u
driver.refresh()	Osvežava trenutnu stranicu

Lociranje elemenata HTML stranice

- Putem
  - vrednosti atributa id
  - vrednosti atributa name
  - teksta u linku
  - dela teksta u linku
  - imena CSS klase
  - CSS selektora
  - XPath upita
  - imena taga



Lociranje elemenata

- *findElement()*
  - kao rezultat vraća WebElement objekat baziran na unetom kriterijumu pretrage
  - izbacuje izuzetak ako ne pronađe element po zadatom kriterijumu pretrage
- *findElements()*
  - kao rezultat vraća listu WebElement objekata koji zadovoljavaju uneti kriterijum pretrage
  - ako ne postoje takvi elementi vraća praznu listu

Java sintaksa za lociranje elemenata

Metod	Sintaksa	Opis
By ID	driver.findElement(By.id(<element ID>))	Lociranje elementa pomoću ID atributa
By Name	driver.findElement(By.name(<element name>))	Lociranje elementa pomoću name atributa
By class name	driver.findElement(By.className(<element class>))	Lociranje elementa pomoću Class atributa
By tag name	driver.findElement(By.tagName(<htmltagname>))	Lociranje elementa pomoću HTML taga
By link text	driver.findElement(By.linkText(<linktext>))	Seletovanje linka pomoću njegovog teksta
By partial link text	driver.findElement(By.partialLinkText(<linktext>))	Lociranje linka pomoću dela njegovog teksta
By CSS	driver.findElement(By.cssSelector(<css selector>))	Lociranje elementa pomoću CSS selektora
By XPath	driver.findElement(By.xpath(<xpath>))	Lociranje elementa pomoću XPath upita

Čekanje

- Često se programski kod testa izvršava brže nego što browser uspeva da prikaže stranicu
- Takođe, često test radi sa podacima za koje je potrebno neko vreme da stignu sa servera
- Potrebno je sinhronizovati rad testa i *browsera*
- Test mora da sačeka da browser učita sve elemente pre nego što nastavi
  - potrebno je zaustaviti test na neko vreme

Implicitno čekanje

- Ako Selenium ne može da pristupi elementu koji test zahteva, sačekaće specificirano vreme i tada pokušati ponovo
  - pre isteka vremena Selenium neće pokušavati da nađe element
  - nakon isteka vremena pokušaće da pronađe element
    - ako pronađe nastavlja dalje
    - ako ne pronađe izbacuje izuzetak
- Potrebno je naznačiti Seleniumu koliko dugo da čeka na element
  - podrazumevana vrednost je 0
  - može se podesiti na neku veću vrednost
  - važi na nivou celog *drivera*
    - ako se postavi, čekanje će se vršiti za sve elemente
  - mana je što čeka i duže nego što je potrebno
- Primer

```
driver.manage().timeouts().
        implicitlyWait(10, TimeUnit.SECONDS);
```

Eksplicitno čekanje

- Može se označiti Seleniumu da čeka do ispunjenja nekog uslova
  - npr. dok element ne bude vidljiv
  - navede se koliko maksimalno da čeka
  - svakih 500 ms proverava da li je uslov ispunjen
- Primer

```
WebElement el = (new WebDriverWait(driver,10))
    .until(ExpectedConditions.presenceOfElementLocated(
        By.id("firstName")));
```

Eksplicitno čekanje

- Može se postaviti i uslov čekanja, ali i frekvencija proveravanja da li je uslov ispunjen
- Koristi se klasa `FluentWait`, pri čemu se navede
  - maksimalno vreme čekanja
  - trajanje perioda nakon kojeg se ponovo proverava ispunjenost uslova
  - koje izuzetke pri proveri ispunjenosti uslova da ignoriše
- Primer

```
Wait<WebDriver> wait = new FluentWait<WebDriver>(driver)
        .withTimeout(60, TimeUnit.SECONDS)
        .pollingEvery(10, TimeUnit.SECONDS)
        .ignoring(NoSuchElementException.class);

WebElement dynamicElement = wait.until(new Function<WebDriver,WebElement>()
{
        public WebElement apply(WebDriver driver) {
                return driver.findElement(By.id("movieForm"));
        }
});
```



What makes a clean test? Three things. Readability, readability, and readability.

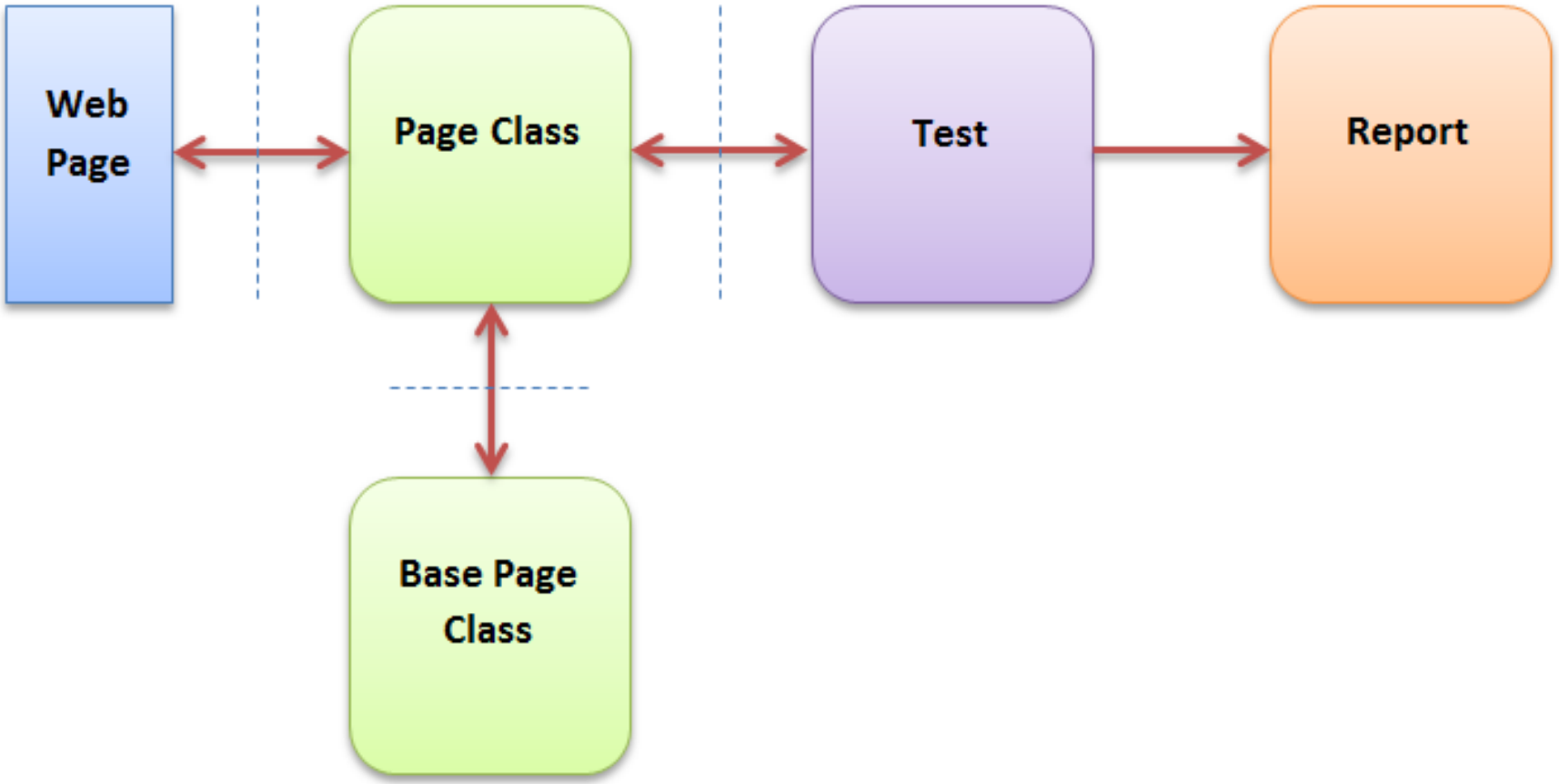
(Clean Code: A Handbook of Agile Software Craftsmanship)

Robert C. Martin
(1952)

PageObject šablon

- Često test pristupa više puta istim elementima
  - tada se isti selektori pišu više puta u kodu
  - pri izmeni izgleda stranice moraju se selektori promeniti na svim mestima
- PageObject šablon razdvaja test od rada sa grafičkim elementima
  - Stranica se modeluje kao objekat neke klase
  - Klasa sadrži elemente koji se nalaze na stranici
  - Klasa sadrži metode za manipulaciju elementima
    - selekcija, preuzimanje vrednosti, postavljanje vrednosti
- Primer
  - `Primeri/selenium/StudentPageTest.java`

PageObject šablon



PageObject šablon

- U testovima se do elemenata dolazi kroz objekat koji predstavlja stranicu
  - test poziva metode *Page* klase
- Test ne sadrži lokatore
  - test ne vodi računa o tome kako se element preuzima
  - lokatori se nalaze u *Page* klasi
- Page klasa ne sadrži *assert* metode
  - logika testiranja je u samom testu
  - *Page* klasa samo obezbeđuje elemente nad kojima se vrše akcije od kojih zavisi uspešnost testiranja
- Ovim je olakšano održavanje testova
  - test slučaj manje zavisi od izgleda stranice
  - manje izmena pri promeni izgleda stranice

Selenium podrška za PageObject šablon

- Klasa PageFactory injektuje *driver* u šablon
  - objekat klase se mora instancirati preko PageFactory klase

```
HomePage hp = PageFactory.initElements(driver,
    HomePage.class);
```

- Lociranje elemenata se može automatski izvršiti stavljanjem `@FindBy` anotacije iznad WebElement objekta u modelu stranice
  - parametar anotacije je selektor za lociranje elementa

```
@FindBy(id = "firstName")
private WebElement inputFirstName;
```